Práctica 3 · Un hilo por cliente: el servidor de eco multihilo

Tiempo estimado: 3 h · Entrega voluntaria: código y respuestas breves, por Moodle · curso 2026-27

Objetivo

Al terminar la práctica sabrás convertir un servidor que atiende a un cliente en uno que atiende a varios a la vez, dedicando un hilo a cada conexión aceptada, y explicar qué cambia y qué no cambia cuando lo haces.

Material: transparencias «Sockets en Java» y «Hilos en Java» (Moodle) · el código de la práctica 2 · JDK 17 o posterior.

Qué hay que hacer

El **cliente** es exactamente el de la práctica 2: no cambia nada en el lado del cliente.

El **servidor de eco multihilo** se ejecuta en `localhost:9999` y acepta tres conexiones y no más. Después de aceptar la tercera, cierra su `ServerSocket`: no entran clientes nuevos, pero los tres hilos de servicio siguen atendiendo a los suyos hasta que cada cliente cierra. Por cada conexión aceptada, el servidor crea un hilo que hace eco de las líneas de ese cliente hasta que lee el fin de fichero (`readLine() == null`); entonces cierra el socket y el hilo termina. Cada hilo se comporta igual que el servidor monohilo de la práctica 2.

Para que el código del servidor quede sencillo, escribe una clase `ServicioEco` que implemente `Runnable`: su método `run()` hace el eco sobre el socket que se le ha dado, hasta el fin de fichero.

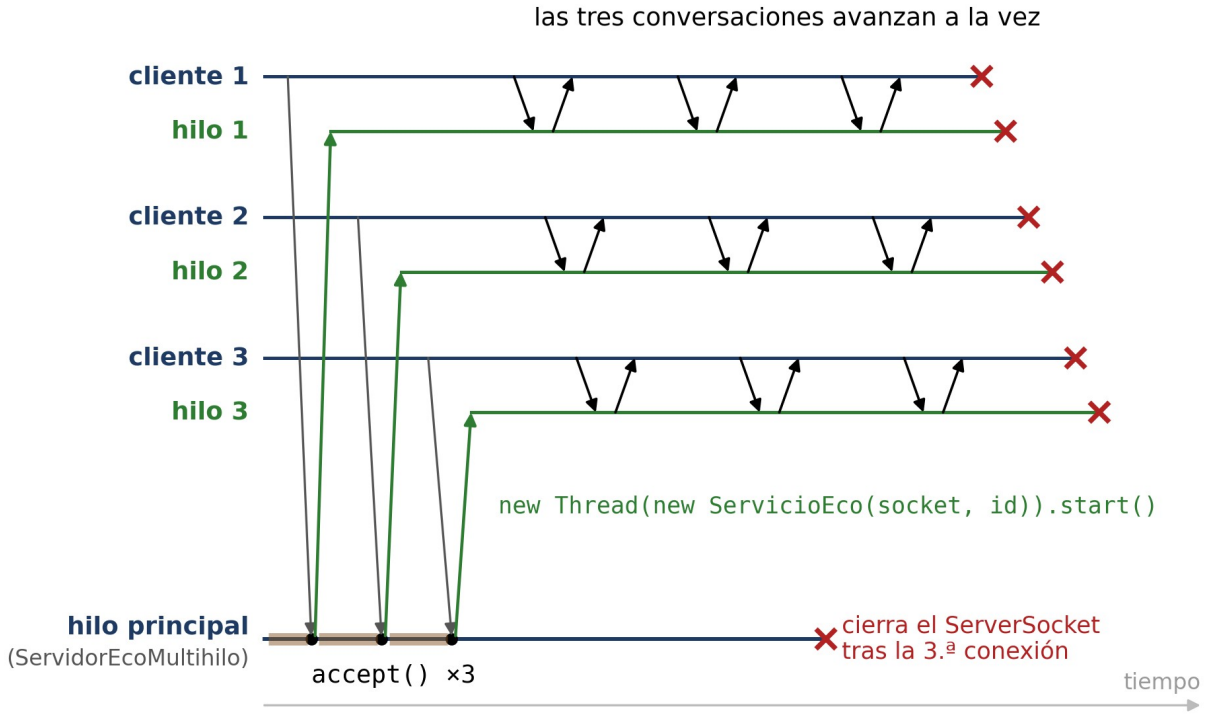


Figura 1. El hilo principal sólo acepta conexiones: por cada una arranca un hilo que ejecuta un ServicioEco y conversa con su cliente, mientras el principal vuelve a accept(). Tras la tercera conexión cierra el ServerSocket.

Esqueleto de la tarea

```
public class ServicioEco implements Runnable {

    private final Socket socket;
    private final int id;

    public ServicioEco(Socket socket, int id) {
        // 1. Guarda el socket del cliente y el número de conexión
    }

    @Override
    public void run() {
        try (
             // 2. El socket, para que se cierre al terminar
             // 3. Un BufferedReader sobre el flujo de entrada del socket
             // 4. Un PrintWriter sobre el flujo de salida, con autovaciado
            ) {

            // 5. Avisa en la consola del servidor de que este cliente
            // ha entrado, con su id
            // 6. Hasta el fin de fichero: lee una línea,
            // imprímela con su id y devuélvela
            // 7. Al salir del bucle, avisa de que el cliente ha cerrado

        } catch (IOException e) {
            System.out.println("[" + id + "] " + e.getMessage());
        }
    }
}
```

Esqueleto del servidor

```
public class ServidorEcoMultihilo {

    public static final int PUERTO = 9999;
    public static final int MAXIMO = 3;

    public static void main(String[] args) {
        try (/* 1. Un ServerSocket escuchando en PUERTO */) {

            int conexiones = 0;
            // 2. Mientras no se llegue a MAXIMO: acepta una conexión, cuéntala
            // y arranca un hilo nuevo que ejecute un ServicioEco sobre ella
            // 3. Al salir del bucle, avisa de que se ha alcanzado el máximo

        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
        // 4. Al salir se cierra el ServerSocket: no entran clientes nuevos,
        // y los hilos de servicio siguen hasta que sus clientes cierran
    }
}
```

Comprobaciones y preguntas

1. Arranca el servidor y tres clientes a la vez, en tres terminales o en tres configuraciones de ejecución. Escribe líneas alternando entre los tres y comprueba en la consola del servidor que los mensajes de los tres hilos se entremezclan.
2. Arranca un cuarto cliente cuando los tres primeros estén conectados. ¿Qué ocurre y por qué? Prueba después a arrancar los cuatro a la vez, y repítelo varias veces: de cuando en cuando el cuarto se conecta sin error y aun así no lo atiende nadie. Explica por qué esa vez fue distinta.

3. Cierra uno de los tres clientes y comprueba que los otros dos siguen funcionando. ¿Qué hilo ha terminado, y qué ha pasado con su socket?
4. Sustituye `start()` por `run()` en la línea que lanza el servicio. ¿Cuántos clientes puede atender entonces el servidor a la vez? Explica qué hilo ejecuta el eco en ese caso. Deshaz el cambio después.
5. El contador `conexiones` lo lleva un solo hilo, el principal. ¿Haría falta `synchronized` si lo incrementaran los hilos de servicio? Razona la respuesta con lo que viste en la práctica 1.

Entrega

Entrega voluntaria, sin nota. Si quieres que te la corrija, sube a la tarea de Moodle tus ficheros `.java` (o un zip del proyecto) y las respuestas breves a las preguntas 2 a 5. La solución se publica después de la fecha de entrega.

Nota de la edición 2026-27

Mismo ejercicio que en cursos anteriores, con correcciones y bajo la numeración de este curso (era la práctica 4.2). El ejercicio 2 de las transparencias 4-Sockets de cursos anteriores (Hernández-Peñaloza y Serrano, 2024; Castilla e Iglesias, 2025, en la sección «Materiales de cursos anteriores» de Moodle) hablaba de «hasta 4» conexiones; el número que vale es el de este enunciado, 3. La solución publicada en cursos anteriores aceptaba en realidad una sola conexión, por un error en la constante. Los identificadores de esta edición están en castellano; la edición en inglés conserva los suyos, y cada una tiene su propia solución.